

AgentQO: Reliability-Aware Joint Logical and Physical Optimization for Multi-Agent LLM Workflows on Multi-GPU Clusters

Shusmoy Chowdhury

Abstract

Multi-agent large language model (LLM) applications are usually designed and served in two separate stages. A workflow designer decides which agents, branches, verifiers, and reasoning rounds should exist. A serving system then schedules that workflow on GPUs. This separation is increasingly limiting because the value of additional agent work depends on both the task state and the current cluster state. A fifth reasoning branch may be useful when earlier branches disagree, but wasteful when they already agree. A verifier may be important for an early planning node whose error affects the rest of the workflow, but less useful for a low-impact formatting step. At the same time, these decisions interact with GPU contention, KV-cache residency, model placement, and communication cost.

This research plan proposes AgentQO, a runtime optimizer that will study logical workflow adaptation and physical scheduling as one problem. The central idea is to estimate the downstream importance of each agent call, which I call epistemic criticality, and use that signal together with a learned interaction cost model. AgentQO will investigate four connected mechanisms: a bidirectional predictor for the latency externality of admitting a call to a busy GPU, a criticality model for estimating the expected task impact of an error, runtime plan edits such as changing redundancy or verification, and value-aware management of KV state and speculative branches. The goal is not to assume that larger workflows are always better. The goal is to decide when extra work is worth its physical cost and where that work should run. I plan to evaluate the idea first in an agent-serving simulator and then on a multi-GPU testbed, with separate ablations for prediction, reliability, logical adaptation, KV management, and speculative cancellation.

Index Terms: LLM serving, multi-agent systems, GPU scheduling, workflow optimization, KV cache, reliability, speculative execution.

I. INTRODUCTION

LLM applications are moving from single requests to multi-step programs that combine model calls, tools, retrieval, memory, and other agents. Common patterns include reason-and-act loops [1], self-consistency [2], tree search [3], debate [4], and iterative refinement [5]. These workflows can improve task performance, but they also multiply inference cost and create dependencies that request-level serving systems were not designed to see.

Modern serving systems already solve several important parts of this problem. vLLM improves memory efficiency through PagedAttention [6]. Orca introduced iteration-level scheduling for generative models [7]. Sarathi reduces prefill interference through chunked prefills [8]. DistServe and Splitwise separate prefill and decode execution [9], [10]. SGLang exploits repeated structure and prefix reuse through RadixAttention [11]. These mechanisms are strong building blocks, but the request is still normally treated as work that must be executed once it arrives.

Agentic workloads add a different question: should all of the requested work exist in the first place? A self-consistency stage can change from three branches to five. A debate can stop after one round or continue. A verifier can be added to one node and omitted from another. A low-value branch can be canceled after enough evidence has already been collected. Recent evaluations also show that adding agents does not reliably improve accuracy and can move a system to a worse accuracy-cost point [12], [13]. At the same time, failure studies show that errors in multi-agent systems are not uniform. Some mistakes propagate through later steps and cause much larger workflow-level damage [14], [15].

This suggests that the amount of agent work should be treated as a runtime optimization variable rather than a fixed input. However, logical adaptation alone is not enough. The value of another branch depends on its physical cost at that moment. A useful verifier may still be a poor decision if it delays several urgent workflows. A branch with shared prefix state may be cheap on one GPU and expensive on another. Moving a call may reduce queueing but lose useful KV state. These are joint logical and physical decisions.

I propose AgentQO as a research direction for this joint problem. AgentQO will combine two ideas from the two earlier versions of this plan. The first is reliability-aware scheduling: physical resources should depend partly on how much an error at a node can affect the final answer. The second is runtime workflow adaptation: the optimizer should be able to change selected parts of the plan when the expected benefit of extra work no longer justifies its cost.

The strongest nearby work already covers parts of this space. SAGA, Autellix, ThunderAgent, Pythia, Maestro, and related systems make the serving layer increasingly aware of programs and workflows [16], [17], [18], [19], [20]. INFRAMIND goes further by adapting planning and reasoning to live infrastructure state [21]. GraphFlow dynamically instantiates workflows and manages state using a graph abstraction [22]. LAMaS learns latency-aware execution graphs and can eliminate redundant future interactions at runtime [23]. Sherlock identifies fragile workflow nodes and selectively adds verification [24]. These systems make a broad claim of “joint optimization” too weak. The research question must be more specific.

The specific question I plan to study is whether a serving optimizer can use *downstream epistemic importance* as a common signal for both logical and physical decisions. That signal would affect whether to add or remove work, which model should execute it, where it should run, which KV state should remain resident, and when speculative work should be stopped. At the same time, a bidirectional cost model would estimate not only how slow a candidate call will be on a GPU, but also how much admitting that call will slow the work already running there.

a) Planned contributions: This research plan has five planned contributions.

- I will define a workflow-level value signal, called epistemic criticality, that estimates the expected final-task impact of an error at an agent node.
- I will adapt the bidirectional interaction-prediction idea used by IconqSched [25] to multi-agent LLM serving, so that admission and placement decisions account for both the candidate call and the calls it may disturb.
- I will formulate runtime plan edits and physical scheduling as one constrained optimization problem rather than two disconnected stages.
- I will investigate value-aware KV residency and an index-style stopping policy for redundant branches, using the same workflow value signal instead of separate heuristics.
- I will evaluate each mechanism independently before evaluating the complete optimizer, using strong single-agent, workflow-serving, infrastructure-aware, reliability-aware, and topology-optimization baselines.

II. PROBLEM SETTING AND MOTIVATION

A multi-agent workflow can be represented as a directed graph $W = (V, E)$. Each node may be an LLM call, tool call, verifier, router, or aggregation step. The graph is only partly known in many applications because later calls depend on earlier outputs and tool results. The serving system therefore observes a sequence of scheduling events rather than one complete static DAG.

For each LLM node, the physical serving layer has choices such as model replica, GPU, start time, batching, preemption, migration, KV residency, and possibly model size. A workflow-aware runtime can also expose a limited set of logical choices. Examples include changing the number of redundant samples, adding a verifier, shortening a debate or refinement loop, selecting a smaller model for a low-impact node, or canceling a branch that is unlikely to change the final answer.

The main difficulty is that logical value and physical cost are coupled. Suppose a workflow has already produced four candidate answers. A fifth candidate has some expected marginal quality benefit. That benefit depends on the agreement or disagreement among the first four candidates. Its physical cost depends on queue length, model availability, KV reuse, current batch composition, and interference with other running calls. The decision should therefore compare marginal quality with marginal system cost at the same point in time.

A second difficulty is that errors have different downstream impact. An early planner may determine the inputs to several later agents, while a late formatting call may only change presentation. Treating both calls as equally important can spend expensive verification or large-model capacity in the wrong place. Failure-analysis work and selective-verification systems provide evidence that workflow structure matters for error propagation [14], [15], [24]. AgentQO will test whether that information is useful below the orchestration layer, inside the physical scheduler.

A third difficulty is externality. The runtime of a new call is not the only cost of admitting it. The new call may increase memory pressure, change batch composition, or compete for GPU execution and memory bandwidth. It

can therefore increase the remaining runtime of calls already in service. IconqSched makes this externality explicit for concurrent database queries [25]. I plan to test whether a similar predictor is practical for agent stages under LLM serving.

III. RELATED WORK AND POSITIONING

The relevant literature spans LLM serving, program-aware agent serving, interference prediction, KV management, workflow optimization, multi-agent reliability, and scheduling theory. I organize this section by the decision each line of work makes. The purpose is not to argue that prior systems ignore agent structure. Several recent systems already use it. The narrower question is which signals they use when deciding how much work to execute and how physical resources should be allocated.

A. LLM Serving Foundations

vLLM introduced PagedAttention to reduce KV-cache fragmentation and support high-throughput serving [6]. Orca showed that generative inference benefits from scheduling at iteration granularity rather than treating a request as one monolithic execution [7]. Sarathi-Serve uses chunked prefills to control the throughput-latency tradeoff caused by long prompt processing [8]. DistServe and Splitwise separate prefill and decode because the two phases have different resource behavior and SLO requirements [9], [10]. SGLang exposes structure in LLM programs and uses RadixAttention to reuse shared prefixes [11]. CacheBlend, Mooncake, and KVLink further extend reuse and KV-centric serving across larger memory and storage hierarchies [26], [27], [28]. Aegaeon studies GPU pooling for concurrent model serving [29].

These systems establish the physical substrate for AgentQO. They show that placement, phase separation, batching, and KV management can materially change serving efficiency. They generally do not decide whether an application should execute another reasoning branch, verifier, or refinement step based on its expected contribution to the final answer. In this plan I therefore treat them as execution backends and resource-management baselines, not as direct competitors to the workflow-value component.

B. Program-Aware and Workflow-Aware Agent Serving

Parrot exposes semantic variables so that the serving layer can reason about dependencies and reuse in LLM applications [30]. Autellix treats an agent program as the scheduling context and prioritizes calls using program-level execution history, addressing cumulative waiting and head-of-line blocking across a program [17]. SAGA makes the workflow a first-class scheduling unit. Its Agent Execution Graph captures structure for KV reuse, while session-affinity batching and workflow-level fairness coordinate related requests across GPUs [16]. ThunderAgent similarly introduces a program abstraction and jointly manages LLM requests and tool-related resources, with a scheduler designed to improve KV reuse and reduce memory imbalance [18].

Pythia exploits workflow predictability to move from reactive serving toward proactive resource coordination [19]. Maestro uses agent roles and stage-level predictions for output length and memory demand, then performs scheduling at node, cluster, and workflow levels under GPU constraints [20]. HexAGenT targets workflow structure and hardware heterogeneity when agent DAGs are revealed over time [31]. Kairos and Astraea also study state-aware, low-latency serving for multi-agent workloads [32], [33]. Justitia focuses on fairness and completion time for task-parallel agents rather than only request-level fairness [34]. Helium frames agent serving from a data-systems perspective and is especially relevant to the query-optimization analogy used in this plan [35].

This line of work establishes that request-level FCFS is not a sufficient abstraction for agents. AgentQO will build on that conclusion rather than restate it. The planned difference is the optimization signal. Most workflow-serving systems optimize latency, throughput, fairness, cache reuse, or SLO attainment for work that is already present in the program. AgentQO will test whether downstream task importance can be used to decide both physical allocation and selected changes to the work itself.

C. Infrastructure-Aware and Adaptive Orchestration

INFRAMIND is an important boundary for this proposal because it already connects orchestration decisions to infrastructure state. It conditions planning on load and budget, changes topology and roles, selects models and

reasoning depth during execution, and schedules queues with an infrastructure-aware policy [21]. This means the research gap cannot be stated as simply “logical and physical decisions have never been joined.”

LAMaS also reduces this gap from the orchestration side. It learns latency-aware execution graphs using critical-path information and includes a runtime controller that can remove redundant future interactions [23]. GraphFlow dynamically instantiates workflows from a graph representation and uses graph structure for workflow-state and KV management [22]. These systems show that runtime workflow adaptation is already becoming part of the serving problem.

AgentQO will therefore focus on a different axis. I plan to make node-level downstream error impact explicit and shared across logical adaptation, physical placement, and state management. I also plan to model the scheduling externality that a candidate call imposes on already-running calls. The evaluation will include INFRAMIND, LAMaS, and GraphFlow wherever reproducible implementations or equivalent policies are available. A useful result would be evidence that these two additional signals improve the quality-cost-latency tradeoff beyond infrastructure-aware topology or depth adaptation alone.

D. Performance Prediction and Interference-Aware Scheduling

The database literature provides a useful model for this part of the plan. Stage predicts query execution time from query-plan features for Amazon Redshift [36]. Graph-based concurrent-query predictors model interactions among simultaneously executing queries [37]. IconqSched goes further by predicting the runtime of a candidate query under the current concurrent state and re-predicting running queries to estimate how their runtimes change if the candidate is admitted [25]. This two-sided view is important because admission has an externality.

LLM serving has developed related prediction mechanisms. SwarmX trains scheduling-specific predictors over prompt, device, runtime, and model features, and uses distributional predictions for low-tail-latency scheduling [38]. Chimera predicts model confidence, remaining workflow length, and congestion to route stages across heterogeneous models [39]. HyGen predicts latency and profiles interference so online and offline workloads can be co-located without violating online SLOs [40]. Harli studies SLO-aware co-location of inference with PEFT workloads [41]. Roomie predicts kernel-level interference for colocated model serving [42]. Earlier datacenter systems such as Paragon, Bubble-Up, PARTIES, Prophet, Horus, and iSAC++ establish a broader lineage of interference-aware placement and QoS control [43], [44], [45], [46], [47], [48].

These works motivate the proposed AgentIconq component. I do not plan to copy a database predictor directly. LLM serving exposes different features, including prefill and decode phases, token counts, batch state, KV residency, model identity, tensor-parallel placement, and shared prefixes. The main idea to test is the two-sided prediction target: estimate both the candidate’s latency and the delay imposed on existing work. I will also test whether positive reuse, such as shared model weights or prefix state, can be represented alongside negative interference.

E. KV Cache and State Management

KV state is a major source of both reuse and memory pressure in agent workloads. KVFlow represents agent execution as a step graph and uses predicted proximity to future activation for fine-grained eviction and prefetching [49]. TokenCake manages KV around function-call gaps, when retaining inactive state may waste GPU memory but premature eviction may cause expensive recomputation [50]. GraphFlow uses workflow structure to manage reusable state [22]. CacheBlend, Mooncake, and KVLink address reuse and movement in broader LLM-serving settings [26], [27], [28]. Work on moving the query rather than the cached state studies the tradeoff between routing computation and transferring latent state across GPU fabrics [51]. AAFLOW provides a complementary data-plane view, reducing communication and serialization cost across agentic pipeline operators [52].

Current KV managers usually prioritize reuse probability, recency, activation timing, locality, or movement cost. AgentQO will test an additional factor: the expected workflow value of the state owner. If two KV regions have similar reuse probability and recomputation cost, preserving the state of a high-criticality planner may be more useful than preserving the state of a branch that is likely to be pruned. This is a hypothesis, not an assumed result. The evaluation will compare value-aware eviction against LRU, predicted-next-use policies, and workflow-aware policies such as KVFlow.

F. Logical Workflow Design and Search

GPTSwarm represents language-agent systems as optimizable computational graphs and improves both node prompts and graph connectivity [53]. AFlow formulates workflow generation as search over code-represented workflows and uses execution feedback to improve the graph [54]. These methods show that agent topology is a meaningful optimization space rather than only an application-development choice.

Their main optimization loop is different from a live multi-tenant GPU scheduler. Offline graph search can spend substantial time evaluating candidate workflows and can optimize against benchmark-level quality and cost. A serving scheduler must make decisions within the execution path and with incomplete knowledge of future arrivals. AgentQO will therefore use a much smaller edit space at runtime. I plan to start with edits that have clear semantics and cheap evaluation: change redundancy, add or remove verification, stop or extend a loop, select model fidelity, and cancel a speculative branch.

G. Reliability, Verification, and the Value of Additional Agents

MAST provides a taxonomy of multi-agent failure modes and shows that failures can arise from specification, inter-agent coordination, verification, and termination [14]. Related work on where agents fail further motivates studying error propagation rather than treating every node as equally risky [15]. AgentForesight moves failure analysis online by auditing partial trajectories and attempting to identify decisive failures before the workflow completes [55].

Sherlock is the closest work to the reliability part of this proposal. It uses counterfactual analysis to identify error-prone workflow nodes, attaches verification selectively, and uses speculative downstream execution to reduce the latency cost of verification [24]. AgentQO shares the idea that verification should be selective. The planned distinction is that epistemic criticality will be used as a general resource-allocation signal, not only as a decision about where to verify. It may influence model size, placement, KV protection, redundancy, and cancellation.

Two recent studies also make the value question unavoidable. A strong single-agent baseline can match homogeneous multi-agent workflows in several settings while benefiting from KV reuse [12]. Controlled protocol-aligned evaluation also finds that adding more agents frequently increases cost without a consistent accuracy gain [13]. These results do not imply that multi-agent workflows are unnecessary. They imply that redundancy should earn its cost. AgentQO is designed around that requirement.

H. SLO and Value-Oriented Scheduling

Tempo defines service gain to represent how useful completion is relative to an application’s SLO and schedules requests using service-gain density [56]. Chimera combines confidence and latency considerations in heterogeneous routing [39]. INFRAMIND balances quality and latency through infrastructure-aware orchestration [21]. These systems are important because they already move beyond raw throughput.

The value signal in this plan is different from deadline utility or local model confidence. Confidence estimates how likely a model is to be correct at a node. Epistemic criticality is intended to estimate how much the final workflow outcome depends on that node being correct. A moderately uncertain node with large downstream reach may therefore deserve more protection than a very uncertain node whose output is one of many interchangeable alternatives. I plan to test whether this distinction is measurable and useful.

I. Scheduling Theory and Simulation

Redundant branches create a stopping problem because both remaining execution time and future value are uncertain. Gittins-index scheduling provides a principled foundation for jobs with uncertain service characteristics [57], while SOAP provides a general framework for age-based scheduling policies [58]. I will use these as theoretical guidance rather than assume their classical optimality conditions hold directly for agent workflows.

For evaluation, AGENTSERVESIM is relevant because it preserves program identity, tool gaps, routing locality, and KV residency across memory tiers, and reports close agreement with real serving behavior [59]. I plan to extend a simulator of this kind with a quality channel so that a logical edit changes not only execution time but also the probability distribution over workflow outcomes. Simulator results will be used to narrow the policy space before hardware experiments.

J. Research Gap

The literature now contains strong solutions for workflow-aware scheduling, infrastructure-aware orchestration, workflow design, interference prediction, KV management, and selective verification. The gap I plan to test is the combination of three signals and decisions:

- 1) **Downstream epistemic importance.** The scheduler estimates how much an error at a node can affect final task quality.
- 2) **Two-sided physical cost.** The scheduler estimates both the cost of a candidate call and the delay that the candidate may impose on existing calls.
- 3) **Shared logical-physical action space.** The same optimizer uses these estimates to decide selected workflow edits, model fidelity, placement, KV residency, redundancy, and cancellation.

I will treat this as a falsifiable research question. If epistemic criticality cannot be predicted cheaply or does not improve scheduling decisions, the evaluation should show that. The project is still useful in that case because the bidirectional cost model, fixed-plan reliability-aware scheduling, value-aware KV policy, and branch-stopping rule can be evaluated independently.

TABLE I: Positioning of representative related work.

Work	Main decision	Useful idea for this plan	Remaining question for AgentQO
vLLM, Orca, Sarathi [6], [7], [8]	Request execution	Memory-efficient KV, iteration scheduling, chunked prefill	Can workflow value guide which work receives these resources?
DistServe, Splitwise [9], [10]	Prefill/decode placement	Phase-aware disaggregation	Can criticality affect model and phase placement?
SGLang and KV-centric serving [11], [26], [27], [28]	Prefix and state reuse	Reuse-aware execution	Should state retention depend partly on downstream task value?
Autellix, SAGA, ThunderAgent [17], [16], [18]	Program/workflow scheduling	Program identity, workflow context, reuse, fairness	How should workflow value affect physical allocation and optional work?
Pythia, Maestro, Hex-AGenT [19], [20], [31]	Predictive workflow and cluster scheduling	Workflow predictability, stage resource prediction, heterogeneity	Can two-sided interference and epistemic value improve placement?
INFRAMIND [21]	Infrastructure-aware planning, routing, depth, scheduling	Connects topology and reasoning decisions to live load	Does explicit downstream error impact add information beyond infrastructure state and learned quality-latency control?
LAMaS [23]	Latency-aware workflow graph	Critical-path-aware graph learning and runtime pruning	Can pruning and plan edits be co-decided with GPU externality and KV state?
GraphFlow [22]	Dynamic workflow generation and state management	Shared graph representation for workflow and KV state	Can workflow state be prioritized by epistemic importance?
IconqSched [25]	Concurrent-query admission	Predicts candidate latency and effect on running work	Can this two-sided prediction transfer to LLM phases, batching, and reuse?
SwarmX, Chimera [38], [39]	Predictive agent routing and scheduling	Runtime predictors, confidence, remaining work, congestion	Can local confidence be separated from downstream task criticality?
HyGen, Harli, Roomie [40], [41], [42]	Interference-aware co-location	Explicit interference and SLO protection	Can positive reuse and workflow value be modeled with interference?
KVFlow, TokenCake [49], [50]	KV eviction and movement	Future-use and tool-gap-aware state management	Does value-aware residency reduce recompute or protect important paths?
GPTSwarm, AFlow [53], [54]	Workflow graph optimization	Topology as an optimization variable	Can a small edit space be optimized online under live cluster state?
Sherlock [24]	Selective verification	Counterfactual fragility and targeted reliability	Can the same reliability signal drive placement, fidelity, KV, and cancellation?

continued on next page

Table I continued

Work	Main decision	Useful idea for this plan	Remaining question for AgentQO
MAST, AgentFore-sight [14], [55]	Failure analysis and online auditing	Failure taxonomy and early risk signals	Can these signals be converted into low-overhead scheduling features?
Tempo [56]	SLO-aware service allocation	Service-gain objective	How does workflow epistemic value interact with deadline and SLO utility?

IV. RESEARCH QUESTIONS

I plan to organize the project around five research questions.

RQ1: Can the serving system predict two-sided interaction cost accurately enough for online scheduling?

The first question is whether an AgentIconq-style model can estimate a candidate call’s latency under the current batch and GPU state, and also estimate the candidate’s effect on running calls.

RQ2: Can downstream epistemic criticality be estimated cheaply before a node or branch finishes? The predictor must be substantially cheaper than the work it is trying to prioritize. I will study structural features, model confidence, agreement, verifier signals, task position, dependency reach, and partial-trajectory signals.

RQ3: Does criticality-aware resource allocation improve quality per unit of GPU cost? I will first test this under a fixed workflow. This isolates the reliability signal from the much harder plan-editing problem.

RQ4: When does runtime workflow adaptation outperform a fixed logical plan? I will study a constrained edit set and compare against strong fixed-plan schedulers, offline workflow optimization, and infrastructure-aware runtime orchestration.

RQ5: Can one value model improve several physical decisions without making the runtime unstable or too expensive? The strongest version of AgentQO uses the same value estimate for model fidelity, placement, KV residency, redundancy, and speculative cancellation. This may simplify the system, but it may also create coupled errors. The ablation study will test both possibilities.

V. PROPOSED AGENTQO DESIGN

A. Workflow Representation and Runtime Edit Space

AgentQO will maintain a runtime workflow state W_t containing completed nodes, active nodes, known future dependencies, and optional edit points. I will not allow arbitrary graph rewriting in the first version. The initial edit space will contain operations that can be expressed and validated locally:

- increase or decrease best-of- N or self-consistency width;
- stop or extend a debate or refinement loop;
- add or skip a verifier at a declared verification point;
- choose among approved model-fidelity levels;
- cancel an optional speculative branch.

This restriction keeps the online search small and makes correctness easier to reason about. Offline methods such as GPTSwarm and AFlow can still generate the base workflow [53], [54]; AgentQO will only adapt declared choices during execution.

B. AgentIconq: Two-Sided Interaction Cost

For a candidate call a considered for GPU g , let R_g be the running set and S_t the serving state. I plan to predict

$$\hat{L}_a(g \mid R_g, S_t) \quad (1)$$

for the candidate’s completion cost, and

$$\Delta_{a \rightarrow j} = \hat{L}_j(R_g \cup \{a\}, S_t) - \hat{L}_j(R_g, S_t) \quad (2)$$

for each running call j . The second term measures the externality of admitting a .

Candidate features will include model identity, prompt and predicted output length, prefill/decode phase, current batch composition, HBM pressure, KV-hit information, tensor-parallel placement, GPU type, queue state, and reusable-prefix indicators. I will compare recurrent, graph-based, and lighter gradient-boosted or MLP predictors. The research goal is not a specific neural architecture. The goal is accurate, low-overhead counterfactual prediction.

A simple physical score can then be written as

$$C_{\text{phys}}(a, g) = \hat{L}_a(g) + \sum_{j \in R_g} \Delta_{a \rightarrow j} + \lambda_m C_{\text{move}}(a, g) - \lambda_r C_{\text{reuse}}(a, g). \quad (3)$$

This form allows both harmful contention and beneficial reuse.

C. Epistemic Criticality

I define epistemic criticality as expected workflow-level quality loss attributable to an error at a node. For node v ,

$$\text{EC}_v = P(\text{err}_v \mid x_v) \cdot (\mathbb{E}[Q \mid v \text{ correct}] - \mathbb{E}[Q \mid v \text{ incorrect}]), \quad (4)$$

where Q is the final task-quality measure and x_v represents features available at scheduling time.

This definition separates two ideas. The first factor estimates local error probability. The second estimates downstream consequence. Structural properties such as depth, fan-out, critical-path membership, verifier availability, and dependency reach will be candidate features for the consequence model rather than fixed multipliers in the definition.

I will evaluate several estimators. The simplest will use calibrated model confidence and graph features. Stronger versions may use a small learned auditor, disagreement among redundant samples, or failure-prediction signals similar to online auditing [55]. Counterfactual labels can be created offline by replacing or perturbing node outputs and measuring final-task change, following the general intuition of selective counterfactual reliability analysis [24].

D. Marginal Value of a Logical Edit

For a candidate logical edit e , such as adding a verifier or one more branch, AgentQO will estimate

$$\hat{U}(e \mid W_t) = \hat{Q}(W_t \oplus e) - \hat{Q}(W_t). \quad (5)$$

The edit will also create new physical work with predicted cost $C_{\text{phys}}(e, P)$ under physical plan P . This makes the basic comparison explicit: an edit should survive only when its expected value is large enough relative to its runtime and resource cost.

I plan to start with monotone or near-monotone edits whose semantics are clear. For example, one more self-consistency sample may improve agreement but has diminishing marginal value. One more verifier may reduce error risk but adds latency. This is easier to model than arbitrary agent-graph rewriting.

E. Joint Logical and Physical Optimization

The runtime decision can be written as

$$\max_{e, P} \hat{Q}(W_t \oplus e) - \lambda_L \hat{L}(W_t \oplus e, P) - \lambda_C \hat{C}(W_t \oplus e, P) - \lambda_R \hat{R}(W_t \oplus e, P), \quad (6)$$

subject to memory, model-placement, dependency, and SLO constraints.

Here e may also be a no-op, so AgentQO can keep the current plan. The risk term will depend on epistemic criticality and the protection assigned to important nodes. In practice I do not plan to solve a large combinatorial optimization from scratch at every token. The initial system will use event-driven optimization when a node arrives, completes, returns from a tool call, reaches a verification point, or creates new branches. Candidate edits and placements will be pruned before scoring.

F. Reliability-Aware Physical Allocation

I will first evaluate criticality under a fixed workflow before enabling plan edits. For a high-criticality node, candidate actions may include a stronger model, protected queue priority, replication, selective verification, or placement that reduces predicted interference. For a low-criticality node, candidate actions may include a smaller model, delayed execution, lower scheduling priority, or no replication.

This fixed-plan stage is important because it tests the main scientific claim directly. If criticality does not improve quality per GPU-hour under a fixed plan, there is little reason to use it inside the larger joint optimizer.

G. Value-Aware KV Residency

For a KV region b owned by workflow node v , I will study a value score such as

$$V_{KV}(b) = P(\text{reuse}_b) \cdot C_{\text{recompute}}(b) \cdot (1 + \kappa EC_v). \quad (7)$$

This is only one candidate formulation. It combines future-use probability, recomputation cost, and workflow importance. I will compare it against LRU, next-use prediction, and workflow-aware eviction without the criticality term [49], [50].

The same value can inform state movement. If routing a call to another GPU loses high-value KV state, the optimizer may keep the call local despite a shorter remote queue. If the state belongs to a low-value branch, moving or recomputing may be preferable.

H. Speculative Branch Stopping

Best-of- N , self-consistency, debate, and refinement create work whose marginal value changes as results arrive. I plan to treat each optional branch as a job with uncertain remaining size and uncertain value. A practical stopping score will compare expected marginal quality gain with expected remaining GPU cost:

$$I_b(t) = \frac{\mathbb{E}[\Delta Q_b \mid h_t]}{\mathbb{E}[C_b^{\text{remain}} \mid h_t]}, \quad (8)$$

where h_t is the observed history.

I will investigate index-style scheduling inspired by Gittins and age-based policies [57], [58], but I will not assume classical optimality results transfer directly. The system will keep a branch while its estimated index is competitive with other waiting work and will cancel it when its expected marginal value becomes too small.

VI. RESEARCH PLAN AND MILESTONES

I plan to execute the project in five stages so that each stage can be evaluated independently.

Stage 1: Reproduce and instrument the serving baseline. I will start with a workflow-aware serving backend and reproduce a subset of representative scheduling and KV policies. I will extend AGENTSERVESIM or a similar program-level simulator with explicit quality outcomes and logical edit events [59]. This stage will define traces, metrics, and reproducibility infrastructure.

Stage 2: Build the two-sided interaction predictor. I will collect concurrency traces over different models, prompt lengths, output lengths, batch states, GPU types, and KV-hit conditions. I will train AgentIconq and compare it with single-call latency prediction and pairwise interference baselines. This stage can stand alone as a scheduling contribution if it consistently improves admission or placement.

Stage 3: Build and test epistemic criticality under a fixed plan. I will generate counterfactual labels and train lightweight criticality estimators. The first scheduler will keep the workflow unchanged and only vary physical protection, model fidelity, and verification. This isolates whether criticality is a useful scheduling signal.

Stage 4: Add value-aware KV management and speculative stopping. I will add criticality-aware KV retention and branch-stopping policies. These mechanisms will be tested separately before they are combined.

Stage 5: Enable joint runtime plan edits. The final stage will enable the constrained logical edit space and co-score each edit with physical placement. I will then evaluate the complete AgentQO design on hardware under multi-tenant load and compare it with fixed-plan, topology-adaptive, and verification-first baselines.

VII. EVALUATION PLAN

A. *Experimental Platforms*

The first evaluation layer will be simulation because the joint policy space is large. I will use a program-level simulator that models tool gaps, program identity, routing locality, and KV state [59]. I will add quality outcomes, optional branch events, and logical edits. The second layer will use a multi-GPU testbed with at least two levels of interconnect locality so that migration and remote-state effects can be observed.

B. *Workloads*

I plan to include several workflow structures:

- serial reason-and-act or planner-executor chains;
- parallel researcher or worker fan-out;
- supervisor-worker workflows;
- self-consistency and best-of- N ;
- debate and iterative refinement;
- tool-heavy coding or web-agent traces;
- heterogeneous-model workflows where different roles can use different model sizes.

Where public traces are available, I will reuse traces from representative workflow-serving work such as SAGA, Maestro, Pythia, or related systems [16], [20], [19]. I will also include strong single-agent execution when the task permits it, because recent work shows that this is an important baseline [12], [13].

C. *Baselines*

The baseline set will be grouped by capability rather than by popularity:

- request-level serving: vLLM and SGLang [6], [11];
- workflow-aware serving: Autellix, SAGA, ThunderAgent, Pythia, and representative cluster schedulers [17], [16], [18], [19], [20];
- predictive or heterogeneous routing: SwarmX and Chimera [38], [39];
- infrastructure-aware adaptation: INFRAMIND and LAMaS where reproducible [21], [23];
- workflow design: fixed graphs produced by GPTSwarm or AFlow [53], [54];
- reliability: Sherlock or a selective-verification policy matching its main mechanism [24];
- KV management: LRU, KVFlow, and tool-gap-aware retention [49], [50];
- SLO/value scheduling: Tempo [56].

D. *Metrics*

I will report both system and task metrics. System metrics will include median, P95, and P99 workflow completion time, throughput, SLO attainment, GPU utilization, HBM occupancy, KV hit rate, KV movement, recomputation, and wasted speculative GPU time. Quality metrics will depend on the task benchmark, but every comparison will include quality per GPU-hour or an equivalent quality-cost measure.

Prediction metrics will include absolute error and Q-error for latency prediction, calibration error for local error probability, ranking quality for criticality, and overhead per scheduling event. For logical adaptation I will report how often each edit is chosen, its realized marginal quality, and its realized resource cost.

E. *Ablation and Stress Tests*

The complete system will be compared against the following ablations:

- no externality term in the physical cost model;
- no positive-reuse term;
- no epistemic criticality;

- local confidence in place of criticality;
- fixed workflow with AgentQO physical scheduling;
- fixed KV policy;
- no speculative cancellation;
- offline plan optimization followed by a strong fixed-plan scheduler;
- logical adaptation without physical co-optimization.

Stress tests will vary concurrency, burstiness, workflow width, tool-call duration, model heterogeneity, GPU heterogeneity, KV pressure, prediction error, and unseen workflow topology. I will also inject controlled predictor noise to identify when the optimizer becomes worse than a simpler baseline.

F. Statistical Method

For hardware experiments, I plan to run each stable configuration multiple times and report confidence intervals rather than only a single best run. For simulation, I will use multiple arrival traces or random seeds and report distributions across workloads. I will separate tuning traces from evaluation traces and include unseen concurrency levels and workflow structures to test generalization. When a policy has more tuning freedom than a baseline, I will report that difference and use matched resource or quality constraints where possible.

VIII. RISKS AND FALLBACK DIRECTIONS

The main scientific risk is that epistemic criticality may be too noisy to predict before execution. If that happens, I will test weaker signals such as structural criticality, sample disagreement, realized verifier signals, or online failure auditing. A negative result is still useful if it identifies which reliability signals are not stable enough for physical scheduling.

The second risk is online optimization overhead. A large joint search could erase the latency benefit it is supposed to create. I will therefore keep the edit space constrained, use event-driven decisions, prune infeasible actions early, and measure optimizer time as a first-class metric.

The third risk is overlap with fast-moving related work. INFRAMIND, GraphFlow, LAMaS, and other recent systems already adapt workflow structure or reasoning based on runtime information [21], [22], [23]. For that reason, I will not base the contribution on the broad statement that the workflow should change at runtime. The contribution will depend on whether downstream error impact and two-sided interference provide useful information for joint resource and workflow decisions.

The fourth risk is that the best policy may be simpler than the full design. If fixed-plan criticality-aware scheduling captures most of the benefit, I will treat that as the main result rather than force joint graph editing. If value-aware KV eviction or speculative stopping is strong independently, either can become a focused systems contribution.

IX. CONCLUSION

This plan studies a specific gap between agent reliability and GPU scheduling. Existing systems increasingly understand workflows, infrastructure state, cache reuse, and even runtime topology adaptation. The open question I want to test is whether the serving layer should also understand how much each piece of work matters to the final answer.

AgentQO will combine a two-sided interaction cost model with epistemic criticality. I will first test those signals under a fixed workflow, then use them for KV management and speculative stopping, and finally allow a constrained set of logical edits to be co-optimized with physical placement. The project is structured so that the central assumptions can fail cleanly. The intended outcome is evidence about when additional agent work is worth its GPU cost, when important work deserves stronger protection, and whether one workflow-level value signal can improve several serving decisions at once.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [2] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2203.11171>
- [3] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://proceedings.neurips.cc/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract.html>
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 11 733–11 763. [Online]. Available: <https://proceedings.mlr.press/v235/du24e.html>
- [5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 46 534–46 594. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP ’23)*. ACM, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yyu>
- [8] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 117–134. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [9] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [10] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00019>
- [11] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
- [12] J. Xu, A. Koesdwydy, S. Bei, Y. Han, B. Huang, D. Wang, Y. Chen, Z. Wang, P. Wang, P. Li, and Y. Ding, “Rethinking the value of multi-agent workflow: A strong single agent baseline,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.12307>
- [13] Anonymous, “Do more agents help? controlled and protocol-aligned evaluation of LLM agent workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.05670>
- [14] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?” 2025. [Online]. Available: <https://arxiv.org/abs/2503.13657>
- [15] Anonymous, “Where LLM agents fail and how they can learn from failures,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.25370>
- [16] D. Guo, J. Wu, and S. M. Yiu, “SAGA: Workflow-atomic scheduling for AI agent inference on GPU clusters,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.00528>
- [17] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, “Autellix: An efficient serving engine for LLM agents as general programs,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.13965>
- [18] H. Kang, Z. Li, X. Yang, W. Xu, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, “Thunderagent: A simple, fast and program-aware agentic inference system,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.13692>
- [19] S. Yu, J. Shu, Y. Ni, K. Qian, X. Li, Y. Wang, J. Zhang, Z. Xu, S. Yang, L. Zhu, E. Zhai, Q. Lu, J. Xing, Y. Lu, X. Jin, X. Liu, and H. Xu, “Pythia: Exploiting workflow predictability for efficient agent-native LLM serving,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.25899>
- [20] J. Wang, X. Zhou, X. Sun, Y. Zhang, Y. Li, T. Wo, X. Wang, C. Hu, and R. Yang, “Maestro: Workload-aware cross-cluster scheduling for LLM-based multi-agent systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.12950>
- [21] A. Kabir, J. Xue, M. Zheng, and Q. Lou, “INFRAMIND: Infrastructure-aware multi-agent orchestration,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.11440>
- [22] A. Li, S. Yang, F. Chen, T. Xu, P. Li, and Z. Su, “GraphFlow: A graph-based workflow management for efficient LLM-agent serving,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.22566>
- [23] X. Shi, M. Zheng, and Q. Lou, “Learning latency-aware orchestration for multi-agent systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.10560>
- [24] Y. Ro, H. Qiu, Í. Goiri, R. Fonseca, R. Bianchini, A. Akella, Y. Wang, M. Erez, and E. Choukse, “Sherlock: Reliable and efficient agentic workflow execution,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.00330>

- [25] Z. Wu, M. Markakis, C. Liu, P. B. Chen, B. Narayanaswamy, T. Kraska, and S. Madden, “Improving DBMS scheduling decisions with accurate performance prediction on concurrent queries,” *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4185–4198, 2025. [Online]. Available: <https://doi.org/10.14778/3749646.3749686>
- [26] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys ’25)*. ACM, 2025, pp. 94–109. [Online]. Available: <https://doi.org/10.1145/3689031.3696098>
- [27] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, “Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot,” in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. Santa Clara, CA: USENIX Association, Feb. 2025, pp. 155–170. [Online]. Available: <https://www.usenix.org/conference/fast25/presentation/qin>
- [28] J. Yang, B. Hou, W. Wei, Y. Bao, and S. Chang, “KVLink: Accelerating large language models via efficient KV cache reuse,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/c217d123885341853cecbdd2be809983-Abstract-Conference.html
- [29] Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou, “Aegaeon: Effective GPU pooling for concurrent LLM serving on the market,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP ’25)*. ACM, 2025, pp. 1030–1045. [Online]. Available: <https://doi.org/10.1145/3731569.3764815>
- [30] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, “Parrot: Efficient serving of LLM-based applications with semantic variable,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 929–945. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [31] Y. Peng, Y. Jiang, W. Li, X. Xu, K. Zhou, J. Jiang, C. Wang, and B. Yuan, “HexAGenT: Efficient agentic LLM serving via workflow- and heterogeneity-aware scheduling,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.16637>
- [32] J. Chen, J. Shi, Q. Chen, and M. Guo, “Kairos: Low-latency multi-agent serving with shared LLMs and excessive loads in the public cloud,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.06948>
- [33] H. Ni, J. Zhang, G. Li, Z. Wang, R. Wu, C. Zhang, and H. Tan, “Astraea: A state-aware scheduling engine for LLM-powered agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.14142>
- [34] M. Yang, G. Wang, M. Luo, Y. Liu, C. Chen, H. Zhao, Y. Feng, Q. Chen, and M. Guo, “Justitia: Fair and efficient scheduling of task-parallel LLM agents with selective pampering,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.17015>
- [35] N. Wadlom, J. Shen, and Y. Lu, “Efficient LLM serving for agentic workflows: A data systems perspective,” 2026, system introduced in the paper: Helium. [Online]. Available: <https://arxiv.org/abs/2603.16104>
- [36] Z. Wu, R. Marcus, Z. Liu, P. Negi, V. Nathan, P. Pfeil, G. Saxena, M. Rahman, B. Narayanaswamy, and T. Kraska, “Stage: Query execution time prediction in amazon redshift,” in *Proceedings of the 2024 ACM SIGMOD International Conference on Management of Data*, 2024, pp. 280–294. [Online]. Available: <https://doi.org/10.1145/3626246.3653391>
- [37] X. Zhou, J. Sun, G. Li, and J. Feng, “Query performance prediction for concurrent queries using graph embedding,” *Proceedings of the VLDB Endowment*, vol. 13, no. 9, pp. 1416–1428, 2020. [Online]. Available: <https://doi.org/10.14778/3397230.3397238>
- [38] Y. Huang, Y. Ye, G. Chen, W. Su, B. Gong, J. Li, Z. Lu, Y. Deng, X. Sun, L. Xu, and L. Mai, “SwarmX: Agentic scheduling for low-latency agentic systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.21401>
- [39] K. Ni, W. Hua, X. Shi, J. Guo, S. Chang, and T. Chen, “Chimera: Latency- and performance-aware multi-agent serving for heterogeneous LLMs,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.22206>
- [40] T. Sun, P. Wang, and F. Lai, “HyGen: Efficient LLM serving via elastic online-offline request co-location,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: <https://arxiv.org/abs/2501.14808>
- [41] A. Xu, H. Zhao, W. Cui, Q. Chen, Y. Chen, S. Zhang, S. Chen, J. Jiang, Z. Yu, and M. Guo, “Harli: SLO-aware co-location of LLM inference and PEFT-based finetuning on model-as-a-service platforms,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.11729>
- [42] Y. Faye, F. Faticanti, S. Jain, and F. Bronzino, “Roomie: Interference-aware colocation for efficient model serving,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.16784>
- [43] C. Delimitrou and C. Kozryakis, “Paragon: QoS-aware scheduling for heterogeneous datacenters,” in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’13)*. ACM, 2013, pp. 77–88. [Online]. Available: <https://doi.org/10.1145/2451116.2451125>
- [44] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, “Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations,” in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44)*. ACM, 2011, pp. 248–259. [Online]. Available: <https://doi.org/10.1145/2155620.2155650>
- [45] S. Chen, C. Delimitrou, and J. F. Martínez, “PARTIES: QoS-aware resource partitioning for multiple interactive services,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’19)*. ACM, 2019, pp. 107–120. [Online]. Available: <https://doi.org/10.1145/3297858.3304005>
- [46] Q. Chen, H. Yang, M. Guo, R. S. Kannan, J. Mars, and L. Tang, “Prophet: Precise QoS prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’17)*. ACM, 2017, pp. 17–32. [Online]. Available: <https://doi.org/10.1145/3037697.3037700>
- [47] G. F. Yeung, D. Borowiec, R. Yang, A. Friday, R. H. R. Harper, and P. Garraghan, “Horus: Interference-aware and prediction-based scheduling in deep learning systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 88–100, 2022. [Online]. Available: <https://doi.org/10.1109/TPDS.2021.3079202>
- [48] Z. Liu, Y. Cheng, C. Chen, J. Hu, R. Fu, and D. Zhang, “ISACPP: Interference-aware scheduling approach for deep learning training workloads based on co-location performance prediction,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 7, pp. 1591–1607, 2025. [Online]. Available: <https://doi.org/10.1109/TPDS.2025.3577796>
- [49] Z. Pan, A. Patel, Y. Shen, Z. Hu, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, “KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: <https://arxiv.org/abs/2507.07400>

- [50] Z. Bian, F. Wu, T. Ma, and Y. Zhuo, “Tokencake: A KV-Cache-centric serving framework for LLM-based multi-agent applications,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.18586>
- [51] B. Ma, J. Eitzinger, H. Köstler, and G. Wellein, “Move the query, not the cache: Characterizing cross-instance latent attention redistribution across GPU fabrics,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.01502>
- [52] A. K. Sarker, M. Staylor, A. Alsaadi, G. von Laszewski, S. Jha, and G. Fox, “AAFLOW: Scalable patterns for agentic AI workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.02162>
- [53] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, “GPTSwarm: Language agents as optimizable graphs,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 62 743–62 767. [Online]. Available: <https://proceedings.mlr.press/v235/zhuge24a.html>
- [54] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, “AFlow: Automating agentic workflow generation,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.10762>
- [55] Anonymous, “AgentForesight: Online auditing for early failure prediction in multi-agent systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.08715>
- [56] W. Zhang, Z. Wu, Y. Mu, B. Liu, M. Lee, and F. Lai, “Tempo: Application-aware LLM serving with mixed SLO requirements,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.20068>
- [57] J. C. Gittins, “Bandit processes and dynamic allocation indices,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 41, no. 2, pp. 148–164, 1979. [Online]. Available: <https://doi.org/10.1111/j.2517-6161.1979.tb01068.x>
- [58] Z. Scully, M. Harchol-Balter, and A. Scheller-Wolf, “SOAP: One clean analysis of all age-based scheduling policies,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 1, pp. 16:1–16:30, 2018. [Online]. Available: <https://doi.org/10.1145/3179419>
- [59] R. H. Rajib, M. Zheng, and Q. Lou, “AGENTSERVESIM: A hardware-aware simulator for multi-turn LLM agent serving,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.09613>